RKNN3 SDK Quick Start

Document ID: RK-JC-YF-416

Release Version: V1.2.0

Date: 2026-10-22

Confidentiality: □ Top Secret □ Secret □ Internal ■ Public

Disclaimer

This document is provided “as is”. Rockchip Electronics Co., Ltd. (the “Company”) makes no express or implied representations or warranties regarding the accuracy, reliability, completeness, merchantability, fitness for a particular purpose, or non-infringement of any statements, information, or content in this document. This document is provided for reference only as a usage guide.

Due to product version upgrades or other reasons, this document may be updated or modified from time to time without prior notice.

Trademark Statement

“Rockchip”, “瑞芯微”, and “瑞芯” are registered trademarks of the Company and are owned by the Company.

All other registered trademarks or trademarks mentioned in this document are owned by their respective owners.

Copyright © 2026 Rockchip Electronics Co., Ltd.

Except as permitted under fair use, no organization or individual may excerpt, reproduce, or distribute any part or all of this document in any form without the prior written permission of the Company.

Rockchip Electronics Co., Ltd.

Address: Building 18, Area A, Software Park, Tongpan Road, Fuzhou, Fujian, China

Website: www.rock-chips.com

Customer Service Tel.: +86-4007-700-590

Customer Service Fax: +86-591-83951833

Customer Service Email: fae@rock-chips.com

Preface

Overview

This document is intended for first-time users of the RKNN3 SDK. Its goal is to help users complete environment checks, model conversion, on-device deployment, and first inference as directly as possible. For the complete software stack, model development workflow, API descriptions, advanced server capabilities, and troubleshooting, refer to the corresponding main documents.

Intended Audience

This document is primarily intended for the following engineers:

Technical support engineers

Software development engineers

Revision History

Version	Author	Date	Description	Reviewer
V0.1.0	HPC	2025-05-13	Initial version (RK182X)	Vincent
V0.2.0	HPC	2025-08-15	1. Adjusted model loading interface description 2. Added model initialization interface description 3. Adjusted model deployment flowchart 4. Added development board preparation 5. Added environment preparation 6. Added sample program execution 7. Updated development workflow 8. Added common issues	Vincent
V0.3.0	HPC	2025-09-10	Updated LLM session-related interface changes	Vincent
V0.3.0b0	HPC	2025-09-12	1. Fixed some errors in the document 2. Added figures 3. Added OpenAI interface usage instructions	Vincent
V0.4.0b0	HPC	2025-10-18	Added support for the RK1828 platform	Vincent
V0.5.0	HPC	2025-11-29	Added synchronous input and output data	Vincent
V1.0.0	HPC	2026-01-01	1. Updated rkllm3-server usage 2. Updated supported model frameworks 3. Updated model evaluation methods	Vincent
V1.0.4	HPC	2026-05-12	Added support for RK3572	Vincent
V1.1.0	HPC	2026-08-03	1. Updated rkllm3-server usage 2. Updated document structure	Vincent
V1.2.0	HPC	2026-10-22	1. Added macOS support related descriptions	Vincent

Table of Contents

1. Before You Begin

1.1 Guide Objectives
1.2 Select a Deployment Mode
1.3 Select an Example Type
1.4 Supported Platforms and Version Requirements
1.5 Obtain the SDK and Related Repositories
2. Hardware Connection and Checks
2.1 Coprocessor Mode (RK182X)
2.1.1 Hardware Checklist
2.1.2 Introduction to the Development Board and Connection Tools
2.1.3 Connect the Development Board
2.2 Non-Coprocessor Mode (RK3572)
2.2.1 Hardware Checklist
2.2.2 Introduction to the Development Board and Connection Tools
2.2.3 Connect the Development Board
2.3 Check Device Connection Status
3. Host-Side and Board-Side Environment Preparation
3.1 Install RKNN3 Toolkit
3.1.1 Install the Python Environment
3.1.2 Install rknn3-toolkit
3.2 Check the Toolkit Installation
3.3 Install or Check the On-Board RKNPU3 Environment
3.3.1 Install Runtime Libraries and Communication Services
3.3.2 Check Versions
3.4 Install Build Tools
3.4.1 Install CMake
3.4.2 Install a Compiler
3.4.2.1 Confirm the Development Board OS Type and Architecture
3.4.2.2 Install NDK for an Android Development Board
3.4.2.3 Install a GCC Cross Compiler for a Linux Development Board
3.5 Runtime Environment Pre-Check
4. Run the First General-Purpose Model

4.1 Obtain the Example
4.2 Convert the Model
4.3 Compile the Deployment Program
4.3.1 Android
4.3.2 Linux
4.3.3 PC Host
4.4 Run the Demo
4.4.1 Development Board Host
4.4.2 PC Host
4.5 Verify the Output
5. Run the First LLM
5.1 Obtain the Example
5.2 Convert the Model
5.3 Compile and Deploy the Program
5.3.1 Android
5.3.2 Linux
5.3.3 PC Host
5.4 Run the Demo
5.4.1 Development Board Host
5.4.2 PC Host
5.5 Verify the Output
6. Start RKLLM3 Server
6.1 Common Parameters
6.2 Start the Service
6.3 Health Check
6.4 Send a Request Using curl
6.5 Send a Request Using the Python OpenAI Library
6.6 Check the Response
7. Common Startup Issues and Next Steps
7.1 Device Not Recognized
7.2 Device/Coprocessor Connection Issues
7.3 Toolkit Installation Verification Failed
7.4 py_utils Not Found During Model Conversion

7.5 GRQ Quantization Failed

7.6 Board-Side Runtime Version Mismatch

7.7 Documentation Navigation

1. Before You Begin

This guide walks users through the first run of the RKNN3 SDK as directly as possible, covering the minimum steps required for environment checks, model conversion, on-device deployment, and first inference. Model optimization principles, complete API descriptions, advanced server features, and systematic troubleshooting are outside the scope of this guide. Refer to the corresponding documentation for those topics.

This guide covers deployment mode selection, environment pre-checks, running the first general-purpose model and LLM, starting RKLLM3 Server and verifying a request, and troubleshooting common first-run issues.

> **Note:** For the complete development workflow, model conversion principles, accuracy/performance/memory analysis, and related topics, refer to *RKNN3_SDK_Development_Guide* and the corresponding API reference documents. The supported hardware, operating systems, and versions are defined by the compatibility matrix included in the SDK release package and are not duplicated in this guide.

1.1 Guide Objectives

After completing this guide, users should be able to:

1. Verify that the development host, development board, and RKNN3 runtime environment are available;
2. Use RKNN3 Toolkit to convert a general-purpose model and run it on the board;
3. Convert an LLM model and run it on the board;
4. Start `rkllm3-server` and send an OpenAI-compatible API request using `curl`;
5. Navigate to the complete development, API reference, or troubleshooting documentation as needed.

1.2 Select a Deployment Mode

RKNN3 SDK supports coprocessor mode and non-coprocessor mode. The main differences are shown below:

In coprocessor mode, the host handles task scheduling and overall control, while RK1820/RK1828 performs AI inference. By host form, the paths are:

Host Form	Hardware Composition	Communication Service Location	Demo Execution Location
Development Board Host	RK3588/RK3576 + RK182X	Runs on RK3588/RK3576	Pushed to the board and run there
PC Host	Windows/Linux PC connects to RK182X via PCIe/Network/USB; macOS PC connects via USB	Runs on the PC	Runs on the PC

Deployment Mode	Hardware Composition	Operation	Relationship Between Host and NPU
Coprocessor mode (RK182X)	Host SoC + RK1820/RK1828 coprocessor	The host SoC handles task scheduling and overall control, while RK1820/RK1828 performs AI inference	Communicates via PCIe/USB/Ethernet
Non-coprocessor mode (RK3572)	RK3572 SoC	AI inference tasks run directly on RK3572	The NPU is integrated into the SoC and shares memory space with the CPU/GPU

The software frameworks for the two modes are shown below:

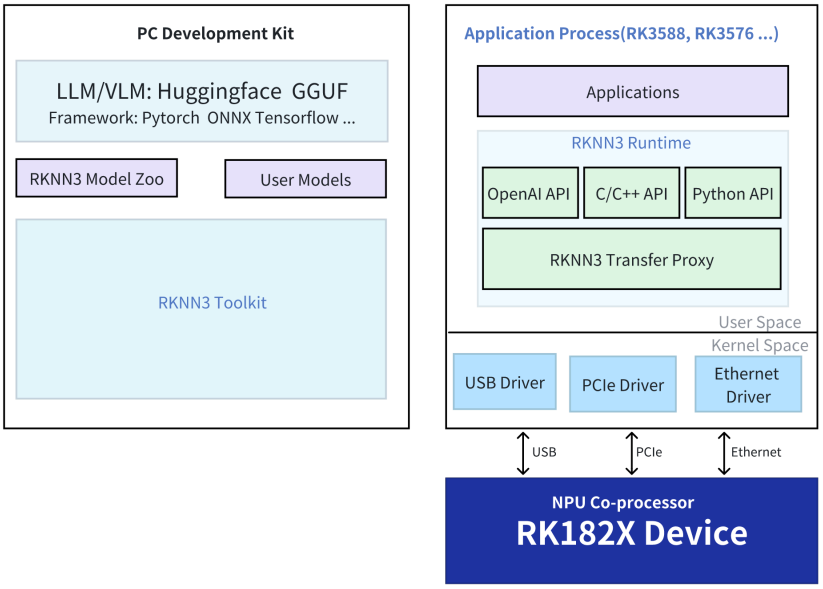


Figure 1-1 RKNN3 SDK Framework (Coprocessor Mode)

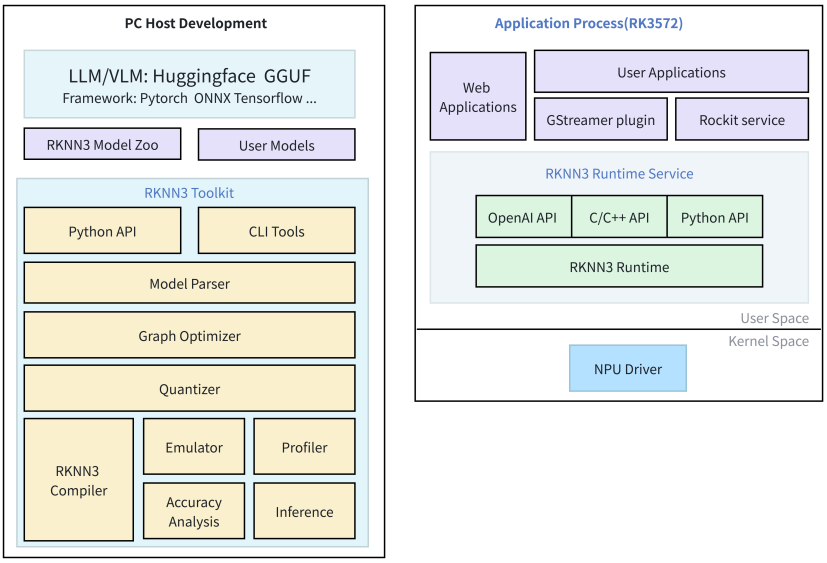


Figure 1-2 RKNN3 SDK Framework (Non-Coprocessor Mode)

The software framework mainly consists of the PC-side development toolkit and the development-board runtime environment.

PC-side development toolkit: Users can use RKNN3 Toolkit to convert models trained with deep learning frameworks such as PyTorch and ONNX into RKNN format. RKNN3 Model Zoo provides model conversion and deployment examples, mainly covering the following model types:

- **CNN models:** MobileNetV2, ResNet50, YOLOv5, YOLOv6, YOLOv8, etc.;
- **LLM models:** Qwen 2.5 0.5B, 1.5B, 3B, etc.;
- **VLM models:** FastVLM, Qwen2.5 VL, Qwen3 VL, etc.

Development-board runtime environment: After model conversion is complete, users can load and run RKNN models on the development board through the RKNN3 API, or invoke LLM models through OpenAI-compatible APIs. The main runtime components are listed below:

Component	Purpose
`RKNN3 API`	Provides core capabilities such as RKNN model loading, inference, LLM model inference, and session management
`rkllm3-server`	Provides an OpenAI-compatible API service supporting text, image, and audio input (video input is not currently supported)
`rknn3_transfer_proxy`	In coprocessor mode, handles communication between the host side and RK1820/RK1828 over PCIe, USB, or Ethernet
`rknn3-server`	Runs on RK3572 in non-coprocessor mode and handles PC-to-board inference requests

The quick-start path is shown below:

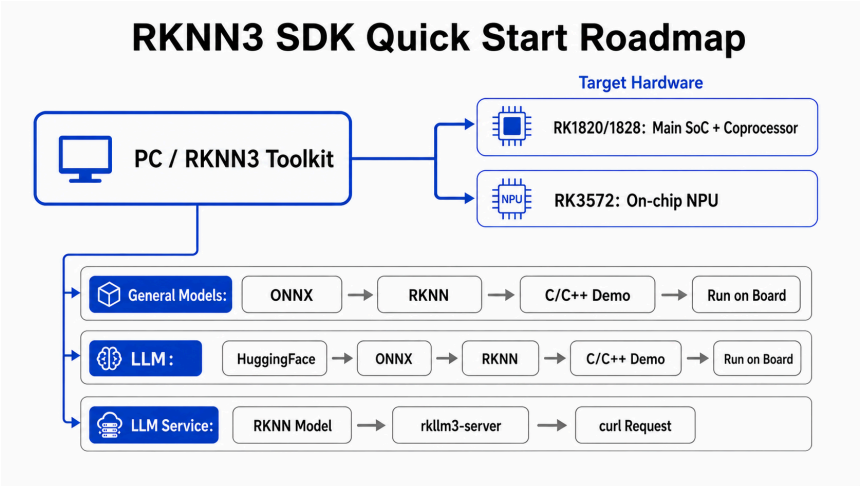


Figure 1-3 RKNN3 SDK Quick-Start Path

For the complete relationships among the software stack components, Runtime responsibilities, and API layers, refer to *RKNN3_SDK_Development_Guide* and related documents.

1.3 Select an Example Type

This guide uses two examples throughout the first-run process:

Example	Purpose	Model Zoo Directory
YOLOv6	First general-purpose model	`examples/yolov6`
Qwen2.5 3B	First LLM	`examples/Qwen2_5`

If your current goal is only to verify the general-purpose model deployment pipeline, completing Chapter 4 is sufficient. If you need to deploy an LLM, continue to Chapter 5. RKLLM3 Server is an independent service entry point; after preparing the LLM model, proceed to Chapter 6.

1.4 Supported Platforms and Version Requirements

Supported hardware platforms are listed below:

Deployment Type	Hardware Platform
Coprocessor platforms	RK3588/RK3576 + RK1820/RK1828 coprocessor
Coprocessor platforms	Windows PC + RK1820/RK1828 coprocessor
Non-coprocessor platform	RK3572

Supported operating systems are listed below:

Deployment Mode	Host Operating System
Coprocessor mode (RK182X)	Android, Linux, Windows, macOS

Deployment Mode	Board-Side Operating System
Non-coprocessor mode (RK3572)	Android, Linux

The development-host examples are based on Ubuntu 20.04 or Ubuntu 22.04. Python 3.10 is recommended for RKNN3 Toolkit. If the current SDK also provides Python 3.12 installation packages, select the version included in the release package.

> **Note:** This guide does not duplicate the complete compatibility information for hardware platforms, host chips, operating systems, Python versions, and component versions. Refer to the shared compatibility matrix in the SDK release package.

1.5 Obtain the SDK and Related Repositories

It is recommended to create a dedicated directory for RKNN3 repositories, for example a folder named `Projects`, and download RKNN3-related repositories into this directory. The SDK can be obtained either from an offline SDK package or from GitHub. The GitHub method is described below.

> **Note:** The directory structure of the SDK offline release package may differ from that of the GitHub repositories. Use the actual directory layout corresponding to the acquisition method you selected.

When using GitHub, download the `rknn3-toolkit` and `rknn3-model-zoo` repositories using the following commands:

```
# Create the Projects directory
mkdir -p ~/Projects

# Enter the directory
cd ~/Projects

# Download the rknn3-toolkit repository
git clone --depth 1 https://github.com/airockchip/rknn3-toolkit

# Download the rknn3-model-zoo repository
git clone --depth 1 https://github.com/airockchip/rknn3-model-zoo
```

The following directory structure is assumed in subsequent examples:

```
Projects
├── rknn3-model-zoo
│   ├── 3rdparty
│   └── examples
├── rknn3-toolkit
│   ├── doc
│   ├── rknn3-toolkit
│   ├── rknn3-toolkit-lite
│   └── rknn3-runtime
│       ├── rknn3-api
│       ├── rknn3_transfer_proxy
│       ├── rknn3-server
│       ├── rkllm3-server
│       └── ...
```

If you use the SDK offline release package, follow its actual directory structure and do not download the GitHub repositories again.

2. Hardware Connection and Checks

This chapter covers only the hardware connections and device visibility checks required for the first run. For hardware principles, complete connection topologies, and additional connection methods, refer to *RKNN3_SDK_Development_Guide* and related documents.

2.1 Coprocessor Mode (RK182X)

Coprocessor mode supports two paths:

- **Development Board Host:** Via RK3588 / RK3576; the development host accesses the board through `adb`
- **PC Host:** Without going through RK3588, `rknn3_transfer_proxy` and the demo run on the PC. Windows / Linux can connect to RK182X via PCIe / Network / USB; macOS currently supports USB only

2.1.1 Hardware Checklist

Development Board Host requires the following hardware:

Hardware	Quantity
Computer	1
RK1820/RK1828 module	1
RK_EVB10_RK3588_V10 development board	1
USB-C data cable	1
RK3588 power adapter	1

> **Note:** In the following sections, the RK1820/RK1828 module is abbreviated as RK1820/RK1828, and the RK_EVB10_RK3588_V10 development board is abbreviated as RK3588.

PC Host requires the following hardware:

Hardware	Quantity
Computer	1
RK1820/RK1828 module	1
USB-C data cable	1
RK182X power adapter	1

2.1.2 Introduction to the Development Board and Connection Tools

RK1820/RK1828 Module



Figure 2-1 RK1820/RK1828 Module

RK_EVB10_RK3588_V10 Development Board



Figure 2-2 RK_EVB10_RK3588_V10 Development Board

USB-C Data Cable



Figure 2-3 USB-C Data Cable

Power Adapter



Figure 2-4 RK3588 Power Adapter

2.1.3 Connect the Development Board

Development Board Host: The following steps use the RK_EVB10_RK3588_V10 development board equipped with RK1820/RK1828 as an example:

1. Prepare a computer running `Ubuntu 20.04` or `Ubuntu 22.04`.
2. Insert the RK1820/RK1828 module into the RK_EVB10_RK3588_V10 development board, as shown below:



Figure 2-5 RK_EVB10_RK3588_V10 Development Board with RK1820/RK1828

1. Connect the port marked as the serial port on the RK1820/RK1828/RK3588 development board to the computer using a data cable. (Note that the interface type and location on the development board may vary depending on the hardware version.)
2. Turn on the power switch and wait until the development board system has fully started.
3. Check whether the development board is connected to the computer.

1) Check whether RK3588 is connected successfully.

In a terminal window on the computer, run the following commands:

```
# Install adb
sudo apt install adb

# List connected adb devices
adb devices
```

When the connection is successful, output similar to the following is displayed. `13af7b28115662cd` is the RK3588 device ID (example value; use the actual value on your system). If no device is displayed, refer to 7.1 Device Not Recognized.

```
List of devices attached
13af7b28115662cd    device
```

2) Check whether RK1820/RK1828 is connected successfully.

In a terminal window on the computer, run the following commands:

```
# Enter the RK3588 shell
adb shell

rknn3_transfer_proxy devices
```

Under normal conditions, the RK1820/RK1828 device should be listed, for example:

```
List of ntb devices attached
0000:01:00.0        b98e6c51    PCIE
```

Note: If `rknn3_transfer_proxy` cannot be found, refer to 3.3 Install or Check the On-Board RKNPU3 Environment.

If `adb devices` shows no device, refer to 7.1 Device Not Recognized. If the host is visible but the coprocessor is not, first refer to the coprocessor checks in 7.1 Device Not Recognized.

PC Host (general):

1. Connect RK182X to the host over USB, power on, and wait until the system has fully started.
2. On the **host**, start the platform-specific `rknn3_transfer_proxy` and run `rknn3_transfer_proxy devices` to confirm that RK182X is visible.
3. You can still install `adb` on the development host (on macOS, use Homebrew, etc.). `adb` is mainly used for Development Board Host or other Android device scenarios; for PC Host, confirm RK182X visibility using `transfer_proxy devices` on the host.

PC Host Platform	Example `transfer_proxy` Path	Example Runtime Library Path
Linux x86_64	`rknn3_transfer_proxy/linux-x86_64/`	`rknn3-api/Linux/x86_64/`
Windows x86_64	`rknn3_transfer_proxy/windows-x86_64/`	`rknn3-api/Windows/x86_64/`
macOS arm64	`rknn3_transfer_proxy/macos-arm64/`	`rknn3-api/macos/arm64/`

PC Host/Linux:

1. Connect RK182X to the Linux host over USB, power on, and wait until startup is complete.
2. (Optional) Install adb for subsequent Development Board Host or other device debugging: `sudo apt install adb`
3. On the host, start the communication service and query devices:

```
cd ~/Projects/rknn3-toolkit/rknn3-runtime
nohup ./rknn3_transfer_proxy/linux-x86_64/rknn3_transfer_proxy > proxy.log 2>&1 &
./rknn3_transfer_proxy/linux-x86_64/rknn3_transfer_proxy devices
```

PC Host/macOS:

1. Install Xcode Command Line Tools:

```
xcode-select --install
```

1. (Optional) Install adb for subsequent Development Board Host or other device debugging:

```
brew install android-platform-tools
adb version
```

1. Connect RK182X to the Mac over USB, power on, and wait until startup is complete.

2. On the Mac host, start the communication service and query devices:

```
cd ~/Projects/rknn3-toolkit/rknn3-runtime
nohup ./rknn3_transfer_proxy/macos-arm64/rknn3_transfer_proxy > proxy.log 2>&1 &
./rknn3_transfer_proxy/macos-arm64/rknn3_transfer_proxy devices
```

Under normal conditions, the RK1820/RK1828 device list should be displayed. If no device is found, check the USB cable, power supply, reconnect the device, and confirm that the proxy is running. The proxy must remain running on the host.

2.2 Non-Coprocessor Mode (RK3572)

2.2.1 Hardware Checklist

The following hardware is required to run the example programs in this document:

Hardware	Quantity
Computer	1
RK3572 development board	1
USB-C data cable	1
RK3572 power adapter	1

> **Note:** In the following sections, the RK3572 development board is abbreviated as RK3572.

2.2.2 Introduction to the Development Board and Connection Tools

RK3572 Development Board



Figure 2-6 RK3572 Development Board

USB-C Data Cable



Figure 2-7 USB-C Data Cable

Power Adapter



Figure 2-8 RK3572 Power Adapter

2.2.3 Connect the Development Board

1. Prepare a computer running `Ubuntu 20.04` or `Ubuntu 22.04`.
2. Connect the port marked as the serial port on the development board to the computer using a data cable. (Note that the interface type and location on the development board may vary depending on the hardware version.)
3. Turn on the power switch and wait until the development board system has fully started.
4. Check whether the development board is connected to the computer.

In a terminal window on the computer, run the following commands:

```
# Install adb
sudo apt install adb

# List connected adb devices
adb devices
```

Under normal conditions, the RK3572 device should be listed, for example:

```
List of devices attached
2f4a8c91e0d3b5a7    device
```

Here, `2f4a8c91e0d3b5a7` is the RK3572 device ID (example value; use the actual value on your system). If no device is displayed, refer to 7.1 Device Not Recognized.

2.3 Check Device Connection Status

Before continuing with installation and model conversion, confirm that the checks below pass for your host form:

Development Board Host / Non-coprocessor mode:

Check Item	Applicable Mode	Check Command
ADB tool available	Development Board Host / RK3572	`adb version`
Board-side device visible	Development Board Host / RK3572	`adb devices`
RK1820/RK1828 coprocessor visible	Coprocessor – Development Board Host	Android: `adb shell "/vendor/bin/rknn3_transfer_proxy devices"`
RK1820/RK1828 coprocessor visible	Coprocessor – Development Board Host	Linux: `adb shell "/usr/bin/rknn3_transfer_proxy devices"`

PC Host (Windows / Linux / macOS):

Check Item	Check Command
ADB tool available	`adb version`
RK1820/RK1828 coprocessor visible	On the host: `./rknn3_transfer_proxy/<platform>/rknn3_transfer_proxy devices`
Proxy process	Linux / macOS: `pgrep rknn3_transfer_proxy`

Continue with host-side and board-side environment preparation only after all applicable checks above have passed.

3. Host-Side and Board-Side Environment Preparation

3.1 Install RKNN3 Toolkit

This quick-start guide provides only the recommended Conda + local wheel installation path. For Docker, system Python, multi-Python-version management, and complete dependency descriptions, refer to *RKNN3_SDK_Development_Guide* and related documents.

3.1.1 Install the Python Environment

If **Python 3.10 (recommended) or Python 3.12** is not installed on the system, or if multiple Python versions are present, it is recommended to use Miniforge Conda to create an isolated Python 3.10 environment.

Step 1: Install Miniforge Conda

First, check whether Miniforge Conda is already installed. If it is installed, skip this step.

```
conda -V
# Example output: conda 23.3.1, indicating Miniforge conda version 23.3.1
# If conda: command not found is displayed, Miniforge is not installed
```

If Miniforge Conda is not installed, download the installer using the following command:

```
wget -c https://github.com/conda-forge/miniforge/releases/download/25.3.0-
1/Miniforge3-25.3.0-1-Linux-x86_64.sh
```

Then install Miniforge Conda using the following commands:

```
# Copy to the user home directory
cp /path/to/Miniforge3-25.3.0-1-Linux-x86_64.sh ~

# Make the installer executable
chmod 777 Miniforge3-25.3.0-1-Linux-x86_64.sh

# Run the installer
bash Miniforge3-25.3.0-1-Linux-x86_64.sh
```

Step 2: Create the Python Environment

After installing Miniforge Conda, refresh the environment variables and enter the Conda base environment:

```
# Refresh environment variables
source ~/.bashrc

# After success, the command prompt changes to:
# (base) xxx@xxx:~$
```

Run the following command to create a Python 3.10 environment named `toolkit3`:

```
conda create -n toolkit3 python=3.10
```

Activate the `toolkit3` environment. Subsequent installation of `rknn3-toolkit` will be performed in this environment:

```
conda activate toolkit3
# After success, the command prompt changes to:
# (toolkit3) xxx@xxx:~$
```

> **Note:** Subsequent Python commands are assumed to be run in the `toolkit3` environment.

3.1.2 Install rknn3-toolkit

After activating the `toolkit3` environment, install `rknn3-toolkit` using the local wheel package:

```
cd ~/Projects/rknn3-toolkit/rknn3-toolkit

# Select the matching requirements file for the Python version and processor
architecture
# cpxxx is the Python version number, and x.x.x is the rknn3-toolkit version
number
pip install -r packages/requirements_cpxxx-x.x.x.txt

# Install rknn3-toolkit
# Select the matching wheel package for the Python version and processor
architecture
# x.x.x is the rknn3-toolkit version number, and cpxx is the Python version number
pip install packages/x86_64/rknn3_toolkit-x.x.x-cpxx-cpxx-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl
```

Replace `x.x.x` and `cpxx/cpxxx` with the actual versions included in the current SDK release package.

3.2 Check the Toolkit Installation

Keep the `toolkit3` environment activated and run:

```
python -c "from rknn.api import RKNN; print('RKNN3 Toolkit import OK')"
```

If the following is displayed:

```
RKNN3 Toolkit import OK
```

the basic Toolkit import check has passed.

3.3 Install or Check the On-Board RKNPU3 Environment

Before running C/C++ demos on the board, prepare the Runtime libraries and the corresponding communication service. The purposes of the components are listed below:

Component	Applicable Mode	Description
`librknn3_api.so`	Both modes	Loads RKNN models and invokes the NPU for inference through the corresponding interfaces
`rknn3_transfer_proxy`	Coprocessor mode	Runs on the host and transfers data with RK1820/RK1828 over PCIe, USB, or Ethernet
`rknn3-server`	Non-coprocessor mode	Runs on RK3572 and handles PC-to-board inference requests

3.3.1 Install Runtime Libraries and Communication Services

First enter the `rknn3-runtime` directory:

```
cd ~/Projects/rknn3-toolkit/rknn3-runtime
```

1. Install Runtime Libraries

Android:

```
adb push rknn3-api/Android/arm64-v8a/librknn3_api.so /vendor/lib64/
adb push rknn3-api/Android/arm64-v8a/librknn3_api_rkcp.so /vendor/lib64/
adb push rknn3-api/Android/arm64-v8a/librknn3_api_native.so /vendor/lib64/
```

Linux:

```
adb push rknn3-api/Linux/aarch64/librknn3_api.so /usr/lib/
adb push rknn3-api/Linux/aarch64/librknn3_api_rkcp.so /usr/lib/
adb push rknn3-api/Linux/aarch64/librknn3_api_native.so /usr/lib/
```

2. Install the Communication Service

Coprocessor mode (RK182X), Android:

```
adb push rknn3_transfer_proxy/android-arm64-v8a/rknn3_transfer_proxy /vendor/bin/
adb shell chmod +x /vendor/bin/rknn3_transfer_proxy
```

Coprocessor mode (RK182X), Linux:

```
adb push rknn3_transfer_proxy/linux-aarch64/rknn3_transfer_proxy /usr/bin/
adb shell chmod +x /usr/bin/rknn3_transfer_proxy
```

Non-coprocessor mode (RK3572), Android:

```
adb push rknn3-server/Android/arm64-v8a/rknn3-server /vendor/bin/
adb shell chmod +x /vendor/bin/rknn3-server
```

Non-coprocessor mode (RK3572), Linux:

```
adb push rknn3-server/Linux/aarch64/rknn3-server /usr/bin/
adb shell chmod +x /usr/bin/rknn3-server
```

After installation, run:

```
adb shell sync
```

3. PC Host (Windows / Linux / macOS)

For PC Host, Runtime libraries and the communication service are on the host side; you do not need to `adb push` them to RK182X. Select directories that match the host platform.

Linux:

1. Runtime libraries are provided with the demo install directory or under `rknn3-api/Linux/x86_64`, for example:

```
install/rknn3_model_test_Linux/lib/
# or prebuilt in the SDK
rknn3-api/Linux/x86_64/
```

1. Communication service:

```
cd ~/Projects/rknn3-toolkit/rknn3-runtime
nohup ./rknn3_transfer_proxy/linux-x86_64/rknn3_transfer_proxy > proxy.log 2>&1 &
```

macOS:

1. Runtime libraries are provided with the demo install directory or under `rknn3-api/macos/arm64`, for example:

```
install/rknn3_model_test_macos/lib/
# or prebuilt in the SDK
rknn3-api/macos/arm64/
```

1. Communication service:

```
cd ~/Projects/rknn3-toolkit/rknn3-runtime
nohup ./rknn3_transfer_proxy/macos-arm64/rknn3_transfer_proxy > proxy.log 2>&1 &
```

For Windows, use the corresponding `windows-x86_64` binaries with the same steps.

> **Important (by host path):**
>
> 1. Development Board Host: After pushing files to the development board, make sure to run `sync` (`adb shell sync` or `sync` on the board). The development host can be Linux, macOS, etc., and uses `adb` to access the board.
> 2. Development Board Host: `rknn3_transfer_proxy` on RK3588 can be configured to start automatically at boot; if not configured, start it manually.

3.3.2 Check Versions

The communication service and Runtime libraries should come from the same SDK version. If a component is missing or versions do not match, follow the steps in Section 3.3.1 to install or update them.

1. Check the Runtime Library Version

Android:

```
adb shell 'strings /vendor/lib64/librknn3_api.so | grep -i "librknn3_api version"'
```

Linux:

```
adb shell 'strings /usr/lib/librknn3_api.so | grep -i "librknn3_api version"'
```

2. Check the Communication Service

RK182X:

```
# Android
adb shell '/vendor/bin/rknn3_transfer_proxy'

# Linux
adb shell '/usr/bin/rknn3_transfer_proxy'
```

RK3572:

```
# Android
adb shell '/vendor/bin/rknn3-server -v'

# Linux
adb shell '/usr/bin/rknn3-server -v'
```

PC Host version check (run on the host):

Linux:

```
strings install/rknn3_model_test_Linux/lib/librknn3_api.so | grep -i 'librknn3_api
version'
./rknn3_transfer_proxy/linux-x86_64/rknn3_transfer_proxy
```

macOS:

```
strings install/rknn3_model_test_macos/lib/librknn3_api.dylib | grep -i
'librknn3_api version'
./rknn3_transfer_proxy/macos-arm64/rknn3_transfer_proxy
```

For the complete update procedure, refer to *RKNN3_SDK_Development_Guide* and related documents.

3.4 Install Build Tools

3.4.1 Install CMake

Linux host:

```
# Update package lists
sudo apt update

# Install cmake
sudo apt install cmake
```

macOS host:

```
brew install cmake
# or confirm Xcode Command Line Tools are installed
xcode-select -p
```

3.4.2 Install a Compiler

3.4.2.1 Confirm the Development Board OS Type and Architecture

For convenience, the development board side is referred to as the "board side" in the following sections.

Step 1: Confirm the Board-Side OS Type

Run the following command in a terminal:

```
adb shell getprop ro.build.version.release
```

If the output is a number (the Android version), the board is running Android:

```
adb shell getprop ro.build.version.release

# Output
12
```

Otherwise, the board is running Linux:

```
adb shell getprop ro.build.version.release

# Output
/bin/bash: line 1: getprop: command not found
```

Step 2: Confirm the Board-Side System Architecture

If the board runs Android, query the system architecture from the PC side using:

```
adb shell getprop ro.product.cpu.abi
```

Example output is shown below. `arm64-v8a` indicates the 64-bit ARM ABI (version 8).

```
adb shell getprop ro.product.cpu.abi

# Output
arm64-v8a
```

If the board runs Linux, query the system architecture from the PC side using:

```
adb shell uname -a
```

Example output is shown below. `aarch64` indicates a 64-bit ARM architecture.

```
adb shell uname -a

# Output
Linux rk3588-buildroot 6.1.118 #4 SMP Sat Jul  5 14:44:53 CST 2025 aarch64
GNU/Linux
```

3.4.2.2 Install NDK for an Android Development Board

> **Scope:** This section applies to Android development boards. If the board runs Linux, skip this section.

- **NDK download URL** (r19c recommended): https://dl.google.com/android/repository/android-ndk-r19c-linux-x86_64.zip
- **Extract the Package**

It is recommended to extract the NDK package into the `Projects` folder as follows:

```
Projects
├── rknn3-toolkit
├── rknn3-model-zoo
└── android-ndk-r19c  # This path will be used later when compiling the RKNN3 C
Demo
```

The NDK path is then `Projects/android-ndk-r19c`.

3.4.2.3 Install a GCC Cross Compiler for a Linux Development Board

> **Scope:** This section applies to Linux development boards. If the board runs Android, skip this section.

Because the application runs on the development board, select the appropriate cross-compiler for the board SoC. For a Linux aarch64 system, use the following example:

- Cross-compiler download URL

```
https://dn.odroid.com/compiler/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu.tar
```

- **Extract the Package**

It is recommended to extract the GCC package into the `Projects` folder. For example, for a 64-bit board-side system, store it as follows:

```
Projects
├── rknn3-toolkit
├── rknn3-model-zoo
└── gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu  # This path will be used later when compiling the RKNN3 C Demo
```

The GCC compiler path is then `Projects/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu`.

3.5 Runtime Environment Pre-Check

Before model conversion, it is recommended to complete the following checks in one pass:

Development Board Host pre-check:

```
# Toolkit
conda activate toolkit3
python -c "from rknn.api import RKNN; print('Toolkit OK')"

# Model Zoo
cd ~/Projects/rknn3-model-zoo
ls examples/yolov6 examples/Qwen2_5

# Build tools
cmake --version

# Device
adb devices
```

PC Host pre-check:

Linux:

```
uname -m
cmake --version
adb version   # optional
cd ~/Projects/rknn3-toolkit/rknn3-runtime
./rknn3_transfer_proxy/linux-x86_64/rknn3_transfer_proxy devices
```

macOS:

```
xcode-select -p
cmake --version
adb version   # optional
cd ~/Projects/rknn3-toolkit/rknn3-runtime
./rknn3_transfer_proxy/macos-arm64/rknn3_transfer_proxy devices
```

If you need to run the LLM example, also install the Model Zoo Python dependencies:

```
cd ~/Projects/rknn3-model-zoo
pip install -r requirements.txt
```

4. Run the First General-Purpose Model

This chapter uses YOLOv6 to complete the minimum end-to-end workflow: obtain the model, convert it, compile the program, run it on the board, and verify the results. For the complete general-purpose model development workflow, model evaluation, quantization, and API call sequence, refer to *RKNN3_SDK_Development_Guide* and *RKNN3_Toolkit_Python_API_Reference*.

4.1 Obtain the Example

Enter the YOLOv6 example directory and download the model:

```
cd ~/Projects/rknn3-model-zoo/examples/yolov6/model
bash download_model.sh
```

`download_model.sh` contains download commands for optimized models with the `_rknn3` suffix and standard models without this suffix. Select the model according to the target platform. Example output:

```
--2025-09-15 11:30:44--
https://ftrg.zbox.filez.com/v2/delivery/data/95f00b0fc900458ba134f8b180b3f7a1/exam
ples/yolov6/yolov6n_rknn3.onnx
Resolving ftrg.zbox.filez.com (ftrg.zbox.filez.com)... 180.184.171.46
Connecting to ftrg.zbox.filez.com (ftrg.zbox.filez.com)|180.184.171.46|:443...
connected.
HTTP request sent, awaiting response... 200
Length: 18644871 (18M) [application/octet-stream]
Saving to: './yolov6n_rknn3.onnx'

./yolov6n_rknn3.onnx                          100%[============================>]
17.78M  2.48MB/s    in 7.9s

2025-09-15 11:30:53 (2.26 MB/s) - './yolov6n_rknn3.onnx' saved [18644871/18644871]
```

> **Note:**
>
> 1. Models with the `_rknn3` suffix perform YOLO post-processing (decoding, candidate box filtering and sorting, NMS, etc.) inside the model. Standard models without the `_rknn3` suffix perform post-processing in the application. In RKNN3 Model Zoo, `yolov6n_rknn3.onnx` is optimized for RK182X. For RK3572, use the standard model `yolov6n.onnx`.
> 2. Download the complete PyTorch example project from PyTorch example. After downloading, refer to the export documentation in the project to generate an RKNN3-optimized ONNX model. The download URL can also be found in `download_model.sh`.

4.2 Convert the Model

Activate the Toolkit environment and enter the conversion script directory:

```
conda activate toolkit3
cd ~/Projects/rknn3-model-zoo/examples/yolov6/python
```

Run the corresponding conversion command for the target platform.

RK182X:

```
python convert.py ../model/yolov6n_rknn3.onnx rk1820 i8
```

RK3572:

```
python convert.py ../model/yolov6n.onnx rk3572 i8
```

The command format is `python convert.py onnx_model_path platform [dtype] [output_rknn_path]`. Here, `i8` enables 8-bit quantization. If `dtype` is omitted, the script enables 8-bit quantization by default. If `output_rknn_path` is omitted, `.rknn` and `.weight` files with the same base name are generated in the input model directory.

Note: The help message in `convert.py` lists only `rk1820`, but the `platform` parameter is passed directly to RKNN3 Toolkit, so `rk3572` is also supported.

After conversion, the corresponding `.rknn` and `.weight` files should appear in the model directory. Example output:

```
--> Loading model
I Loading : 100%|████████████████████████████████████████████| 142/142
[00:00<00:00, 24888.89it/s]
W load_onnx: Please note that some float16/float64 data types in the model have
been modified to float32!
```

```
done
--> Building model
I FoldConstant : 100%|████████████████████████████████████████| 218/218
[00:00<00:00, 505.15it/s]
I OpFusing 0: 100%|████████████████████████████████████████| 100/100
[00:00<00:00, 522.77it/s]
I OpFusing 1 : 100%|████████████████████████████████████████| 100/100
[00:00<00:00, 344.86it/s]
I OpFusing 0 : 100%|████████████████████████████████████████| 100/100
[00:00<00:00, 274.72it/s]
I OpFusing 1 : 100%|████████████████████████████████████████| 100/100
[00:00<00:00, 261.63it/s]
I OpFusing 2 : 100%|████████████████████████████████████████| 100/100
[00:00<00:00, 249.86it/s]
I OpFusing 0 : 100%|████████████████████████████████████████| 100/100
[00:00<00:00, 231.67it/s]
I OpFusing 1 : 100%|████████████████████████████████████████| 100/100
[00:00<00:00, 222.40it/s]
I OpFusing 2 : 100%|████████████████████████████████████████| 100/100
[00:00<00:00, 220.71it/s]
I     FoldConstant : 100%|████████████████████████████████████| 170/170
[00:00<00:00, 622.77it/s]
...
I rknn building ...
I Split shape 0 done
I Compile all models for 1 cores
module optimization: [=============================================================] 1/1
(100.0%) Total: 0s
register configuration: [=============================================================]
1/1 (100.0%) Total: 0s
memory optimization: [=============================================================] 1/1
(100.0%) Total: 0s
weight sharing: [=============================================================] 1/1
(100.0%) Total: 0s
code generation: [=============================================================] 1/1
(100.0%) Total: 1s
I rknn building done.
done
--> Export rknn model
done
```

4.3 Compile the Deployment Program

4.3.1 Android

Android uses `build-android.sh` for compilation, with the target architecture `arm64-v8a`.

1. Configure the NDK Path

Set `ANDROID_NDK_PATH` at the beginning of the `build-android.sh` script:

```
ANDROID_NDK_PATH=Projects/android-ndk-r19c
```

2. Compile the demo

```
cd ~/Projects/rknn3-model-zoo

# RK182X: the application runs on the host SoC such as RK3588/RK3576
./build-android.sh -t rk3588 -a arm64-v8a -b Release -d yolov6

# RK3572
./build-android.sh -t rk3572 -a arm64-v8a -b Release -d yolov6
```

3. Deploy the demo

```
# RK182X
adb push install/rk3588_android_arm64-v8a/rknn_yolov6_demo /data/local/tmp/

# RK3572
adb push install/rk3572_android_arm64-v8a/rknn_yolov6_demo /data/local/tmp/
```

> During Android debugging, `/data/local/tmp` is recommended to avoid insufficient permissions on `/data`. If the firmware permits the use of `/data`, adjust the deployment directory according to project requirements.

4.3.2 Linux

Linux uses `build-linux.sh` for compilation, with the target architecture `aarch64`.

1. Configure the Cross Compiler

```
export GCC_COMPILER=~/opts/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-
gnu/bin/aarch64-linux-gnu
cd ~/Projects/rknn3-model-zoo
```

2. Compile the demo

```
# RK182X: the application runs on the host SoC such as RK3588/RK3576
./build-linux.sh -t rk3588 -a aarch64 -b Release -d yolov6

# RK3572
./build-linux.sh -t rk3572 -a aarch64 -b Release -d yolov6
```

3. Deploy the demo

```
# RK182X
adb push install/rk3588_linux_aarch64/rknn_yolov6_demo /data/

# RK3572
adb push install/rk3572_linux_aarch64/rknn_yolov6_demo /data/
```

4.3.3 PC Host

For PC Host, compile natively on the host; you do not need to `adb push` to RK182X.

> **Platform differences:**
>
> | Platform | Repository | Build Script | Executable | Output Directory |
> | --- | --- | --- | --- | --- |
> | Linux x86_64 | `rknn3-model-zoo` | `./build-linux.sh -t x86 -a x86_64 -b Release -d yolov6` | `rknn_yolov6_demo` | `install/x86_64/rknn_yolov6_demo/` |
> | macOS | `rknn3-runtime/examples/rknn3_model_test_demo` | `./build-macos.sh` | `rknn3_model_test` | `install/rknn3_model_test_macos/` |
>
> `rknn3-model-zoo` does not currently support building macOS platform demos.

Linux:

```
cd ~/Projects/rknn3-model-zoo
./build-linux.sh -t x86 -a x86_64 -b Release -d yolov6
# Output directory
install/x86_64/rknn_yolov6_demo/
```

macOS:

```
cd ~/Projects/rknn3-toolkit/rknn3-runtime/examples/rknn3_model_test_demo
./build-macos.sh
# Output directory
install/rknn3_model_test_macos/
```

`build-macos.sh` supports only the optional `-b` (Debug/Release) flag; there is no `-d` parameter.

4.4 Run the Demo

4.4.1 Development Board Host

First enter the board shell:

```
adb shell
```

1. Enter the demo directory and configure the dynamic library path

Android:

```
cd /data/local/tmp/rknn_yolov6_demo
export LD_LIBRARY_PATH=./lib${LD_LIBRARY_PATH:+:$LD_LIBRARY_PATH}
```

Linux:

```
cd /data/rknn_yolov6_demo
export LD_LIBRARY_PATH=./lib${LD_LIBRARY_PATH:+:$LD_LIBRARY_PATH}
```

2. Run the demo

Command format:

```
./rknn_yolov6_demo <model_path> <weight_path> <image_path> <core_mask>
```

RK182X:

```
./rknn_yolov6_demo \
  ./model/yolov6n_rknn3.rknn \
  ./model/yolov6n_rknn3.weight \
  ./model/bus.jpg 1
```

RK3572:

```
./rknn_yolov6_demo \
  ./model/yolov6n.rknn \
  ./model/yolov6n.weight \
  ./model/bus.jpg 1
```

4.4.2 PC Host

First start the communication service:

Linux:

```
cd ~/Projects/rknn3-toolkit/rknn3-runtime
nohup ./rknn3_transfer_proxy/linux-x86_64/rknn3_transfer_proxy > proxy.log 2>&1 &
```

macOS:

```
cd ~/Projects/rknn3-toolkit/rknn3-runtime
nohup ./rknn3_transfer_proxy/macos-arm64/rknn3_transfer_proxy > proxy.log 2>&1 &
```

1. Enter the demo directory and configure the dynamic library path

Linux:

```
cd ~/Projects/rknn3-model-zoo/install/x86_64/rknn_yolov6_demo
export LD_LIBRARY_PATH=./lib${LD_LIBRARY_PATH:+:$LD_LIBRARY_PATH}
```

macOS:

```
cd ~/Projects/rknn3-toolkit/rknn3-
runtime/examples/rknn3_model_test_demo/install/rknn3_model_test_macos
export DYLD_LIBRARY_PATH=./lib
```

2. Run the demo

Linux:

```
./rknn_yolov6_demo <model_path> <weight_path> <image_path> <core_mask>
```

```
./rknn_yolov6_demo \
  ./model/yolov6n_rknn3.rknn \
  ./model/yolov6n_rknn3.weight \
  ./model/bus.jpg 1
```

macOS:

```
./rknn3_model_test <model_path> <weight_path> [core_mask] [input_npy]
[golden_output_npy]
```

```
./rknn3_model_test model.rknn model.weight

# Example with input/output accuracy comparison
./rknn3_model_test model.rknn model.weight 0x1 in.npy out.npy
```

4.5 Verify the Output

When execution succeeds, the terminal should print inference time, object class, confidence score, and bounding box information, for example:

```
model is NHWC input fmt
model input height=640, width=640, channel=3
origin size=640x640 crop size=640x640
input image: 640 x 640, subsampling: 4:2:0, colorspace: YCbCr, orientation: 1
--> inference model
scale=1.000000 dst_box=(0 0 639 639) allow_slight_change=1 _left_offset=0
_top_offset=0 padding_w=0 padding_h=0
rga_api version 1.10.1_[0]
Pre-process time: 2.09 ms
-->rknn_run
Inference time: 21.37 ms
Post-process time: 0.44 ms
Total time: 23.90 ms
--> inference model done
bus @ (97 140 557 440) 0.951
person @ (109 239 221 537) 0.940
person @ (213 238 287 511) 0.932
person @ (480 233 560 520) 0.924
person @ (78 326 117 515) 0.455
write_image path: out.png width=640 height=640 channel=3 data=0x7f885e801
```

To view the result image, run the following command on the development host:

```
# Android example path
adb pull /data/local/tmp/rknn_yolov6_demo/out.png .

# Linux example path
adb pull /data/rknn_yolov6_demo/out.png .
```

For PC Host, result files are generated directly in the host's current directory and can be opened locally. If you use the Development Board Host path, you can still pull results with `adb pull`.

The actual detection results vary with the model version, input image, and runtime environment. For the first run, it is sufficient to confirm that the demo completes successfully and generates `out.png`.

5. Run the First LLM

This chapter uses Qwen2.5 3B to complete LLM model export, RKNN conversion, C/C++ demo compilation, and on-board inference. For complete information about LLM ONNX constraints, KVCache, Session, LoRA, quantization configuration, and related topics, refer to *RKNN3_SDK_Development_Guide* and *RKNN3_Runtime_C_API_Reference*.

5.1 Obtain the Example

Confirm that the Qwen2.5 example exists in Model Zoo:

```
cd ~/Projects/rknn3-model-zoo
ls examples/Qwen2_5
```

Install the example dependencies:

```
conda activate toolkit3
pip install -r requirements.txt
```

If the example requires downloading the original model from Hugging Face, prepare the model path according to the README or default script configuration in `examples/Qwen2_5`. This guide does not duplicate the download procedure for each model.

5.2 Convert the Model

The first LLM run mainly consists of three stages: model export, RKNN conversion, and on-board execution. This section first completes model export and conversion.

First set the Model Zoo Python path:

```
cd ~/Projects/rknn3-model-zoo
export PYTHONPATH="$PWD:$PYTHONPATH"
cd examples/Qwen2_5/python
```

Step 1: Export ONNX and LLM Supporting Files

```
python export_llm.py
```

After successful execution, the `examples/Qwen2_5/model` directory will contain model files with the `.onnx` suffix, configuration files with the `.config.pkl` suffix, and tokenizer and embedding files.

Step 2: Convert to an RKNN Model

RK182X:

```
python export_rknn.py --platform rk1820
```

RK3572:

```
python export_rknn.py --platform rk3572
```

Example output:

```
I rknn building done.
done
--> Export RKNN model
done
```

After the steps above, the corresponding `.rknn` and `.weight` model files should be generated, together with board-side runtime files such as `.tokenizer.gguf` and `.embed.bin`.

5.3 Compile and Deploy the Program

5.3.1 Android

Android uses `build-android.sh` for compilation, with the target architecture `arm64-v8a`.

1. Configure the NDK Path

```
ANDROID_NDK_PATH=Projects/android-ndk-r19c
```

2. Compile the demo

```
cd ~/Projects/rknn3-model-zoo

# RK182X
./build-android.sh -t rk3588 -a arm64-v8a -b Release -d Qwen2_5

# RK3572
./build-android.sh -t rk3572 -a arm64-v8a -b Release -d Qwen2_5
```

3. Deploy the demo

```
# RK182X
adb push install/rk3588_android_arm64-v8a/rknn_Qwen2_5_demo /data/local/tmp/

# RK3572
adb push install/rk3572_android_arm64-v8a/rknn_Qwen2_5_demo /data/local/tmp/
```

5.3.2 Linux

Linux uses `build-linux.sh` for compilation, with the target architecture `aarch64`.

1. Configure the Cross Compiler

```
export GCC_COMPILER=~/opts/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-
gnu/bin/aarch64-linux-gnu
cd ~/Projects/rknn3-model-zoo
```

2. Compile the demo

```
# RK182X
./build-linux.sh -t rk3588 -a aarch64 -b Release -d Qwen2_5

# RK3572
./build-linux.sh -t rk3572 -a aarch64 -b Release -d Qwen2_5
```

3. Deploy the demo

```
# RK182X
adb push install/rk3588_linux_aarch64/rknn_Qwen2_5_demo /data/

# RK3572
adb push install/rk3572_linux_aarch64/rknn_Qwen2_5_demo /data/
```

5.3.3 PC Host

For PC Host, compile natively on the host; you do not need to `adb push` to RK182X.

> **Platform differences:**
>
> | Platform | Repository | Build Script | Executable | Output Directory |
> |---|---|---|---|---|
> | Linux x86_64 | `rknn3-model-zoo` | `./build-linux.sh -t x86 -a x86_64 -b Release -d Qwen2_5` | `rknn_qwen2_5_demo` | `install/x86_64/rknn_Qwen2_5_demo/` |
> | macOS | `rknn3-runtime/examples/rknn3_session_test_demo` | `./build-macos.sh` | `rknn3_session_test` | `install/rknn3_session_test_macos/` |
>
> `rknn3-model-zoo` does not currently support building macOS platform demos.

Linux:

```
cd ~/Projects/rknn3-model-zoo
./build-linux.sh -t x86 -a x86_64 -b Release -d Qwen2_5
# Output directory
install/x86_64/rknn_Qwen2_5_demo/
```

macOS:

```
cd ~/Projects/rknn3-toolkit/rknn3-runtime/examples/rknn3_session_test_demo
./build-macos.sh
# Output directory
install/rknn3_session_test_macos/
```

`build-macos.sh` supports only the optional `-b` (Debug/Release) flag; there is no `-d` parameter. Copy the model files produced in Section 5.2 into the corresponding install directory before running.

5.4 Run the Demo

5.4.1 Development Board Host

First enter the board shell:

```
adb shell
```

1. Enter the demo directory and configure the dynamic library path

Android:

```
cd /data/local/tmp/rknn_Qwen2_5_demo
export LD_LIBRARY_PATH=./lib${LD_LIBRARY_PATH:+:$LD_LIBRARY_PATH}
```

Linux:

```
cd /data/rknn_Qwen2_5_demo
export LD_LIBRARY_PATH=./lib${LD_LIBRARY_PATH:+:$LD_LIBRARY_PATH}
```

2. Run the Qwen2.5 demo

Command format:

```
./rknn_qwen2_5_demo <model_path> <weight_path> <tokenizer_path> <embedding_path>
<core_mask> <prompt>
```

RK182X:

```
./rknn_qwen2_5_demo \
  ./model/Qwen2.5-3B-Instruct.rknn \
  ./model/Qwen2.5-3B-Instruct.weight \
  ./model/Qwen2.5-3B-Instruct.tokenizer.gguf \
  ./model/Qwen2.5-3B-Instruct.embed.bin \
  0xff \
  "who are you?"
```

RK3572:

```
./rknn_qwen2_5_demo \
  ./model/Qwen2.5-3B-Instruct.rknn \
  ./model/Qwen2.5-3B-Instruct.weight \
  ./model/Qwen2.5-3B-Instruct.tokenizer.gguf \
  ./model/Qwen2.5-3B-Instruct.embed.bin \
  0x01 \
  "who are you?"
```

If Android reports `Permission denied` when executing `adb push ... /data`, you can push the demo to `/data/local/tmp` during debugging and update the subsequent `cd` path accordingly.

5.4.2 PC Host

First start the communication service:

Linux:

```
cd ~/Projects/rknn3-toolkit/rknn3-runtime
nohup ./rknn3_transfer_proxy/linux-x86_64/rknn3_transfer_proxy > proxy.log 2>&1 &
```

macOS:

```
cd ~/Projects/rknn3-toolkit/rknn3-runtime
nohup ./rknn3_transfer_proxy/macos-arm64/rknn3_transfer_proxy > proxy.log 2>&1 &
```

1. Enter the demo directory and configure the dynamic library path

Linux:

```
cd ~/Projects/rknn3-model-zoo/install/x86_64/rknn_Qwen2_5_demo
export LD_LIBRARY_PATH=./lib${LD_LIBRARY_PATH:+:$LD_LIBRARY_PATH}
```

macOS:

```
cd ~/Projects/rknn3-toolkit/rknn3-
runtime/examples/rknn3_session_test_demo/install/rknn3_session_test_macos
export DYLD_LIBRARY_PATH=./lib
```

2. Run the demo

Linux:

```
./rknn_qwen2_5_demo <model_path> <weight_path> <tokenizer_path> <embedding_path>
<core_mask> <prompt>
```

```
./rknn_qwen2_5_demo \
  ./model/Qwen2.5-3B-Instruct.rknn \
  ./model/Qwen2.5-3B-Instruct.weight \
  ./model/Qwen2.5-3B-Instruct.tokenizer.gguf \
  ./model/Qwen2.5-3B-Instruct.embed.bin \
  0xff \
  "who are you?"
```

macOS:

```
./rknn3_session_test <rknn_path> <weight_path> <tokenizer.gguf> <embedding.bin>
<max_context_len> <max_new_tokens> <core_mask>
```

```
./rknn3_session_test \
  ./model/Qwen2.5-3B-Instruct.rknn \
  ./model/Qwen2.5-3B-Instruct.weight \
  ./model/Qwen2.5-3B-Instruct.tokenizer.gguf \
  ./model/Qwen2.5-3B-Instruct.embed.bin \
  2048 512 0xff
```

5.5 Verify the Output

For the first run, confirm the following three points:

1. The model, weight, tokenizer, and embedding files can all be loaded successfully;
2. No errors occur during Runtime initialization or Session inference;
3. The terminal displays text generated by the model.

Different model versions and sampling configurations may produce different responses, so the output text does not need to exactly match the example in this document.

6. Start RKLLM3 Server

`rkllm3-server` is a basic LLM server built on RKNPU3. It provides an OpenAI API-compatible chat interface. This chapter demonstrates how to start the service, send requests, and verify results.

The main features are listed below:

Feature	Description
LLM inference	Performs LLM inference using RKNPU3
OpenAI-compatible interface	Provides an OpenAI API-compatible chat interface
Multimodal input	Supports text, image, and audio input; video input is not currently supported

> **Note:** This chapter covers only the minimum steps required to start the service and complete one request. For advanced capabilities such as multi-model loading, multiple sessions, slot pause/resume/interruption, KV Cache save/load, checkpoints, LoRA, Tie Word Embedding, VLM/Audio/Video input, complete API endpoint definitions, and error codes, refer to *RKLLM3_Server_User_Guide* and the Server API Reference/OpenAPI documentation.

6.1 Common Parameters

When starting `rkllm3-server`, you need to specify parameters such as the model file, vocabulary, and embedding file. The most commonly used parameters for this quick-start guide are listed below.

General Parameters

Parameter	Description
`-m, --model FNAME`	LLM model path
`--weight FNAME`	LLM weight path (optional)
`--vocab FNAME`	Vocabulary path
`--embed FNAME`	Path to the embedding binary file
`-c, --ctx-size N`	Prompt context size (default: 4096; 0 = load from model)
`-n, --predict, --n_predict N`	Number of tokens to predict (default: -1; -1 = context size)
`-a, --alias STRING`	Sets the model alias (used by the REST API)
`--core-mask MASK`	NPU core mask used for LLM execution (for example, `0xff` = all cores). Default: auto

Sampling Parameters

Parameter	Description
`--temp N`	Temperature (default: 0.8)
`--top-k N`	Top-k sampling (default: 40; 0 = disabled)
`--top-p N`	Top-p sampling (default: 0.9; 1.0 = disabled)
`--repeat-penalty N`	Penalty for repeated token sequences (default: 1.0; 1.0 = disabled)
`--presence-penalty N`	Presence penalty coefficient for repeated tokens (default: 0.0; 0.0 = disabled)
`--frequency-penalty N`	Frequency penalty coefficient for repeated tokens (default: 0.0; 0.0 = disabled)

Service Parameters

Parameter	Description
`--host HOST`	Listening IP address (default: 127.0.0.1)
`--port PORT`	Listening port (default: 8080)
`-to, --timeout N`	Server read/write timeout in seconds (default: 600)
`--device-id STRING`	Device ID (required to select a specific device when multiple devices are present)
`--log-level N`	Log level (default: 0)
`-h, --help, --usage`	Prints usage information and exits

> For the complete parameter list, including advanced parameters for multimodal input, LoRA, checkpoints, and Tie Word Embedding, refer to *RKLLM3_Server_User_Guide*.

6.2 Start the Service

Before starting the service, select the `rkllm3-server` executable that matches the board operating system from the current SDK's `rknn3-runtime/rkllm3-server/bin` directory, and place it in the same working directory as the Qwen2.5 model files.

1. Prepare Runtime Files

The working directory should contain at least the following files:

```
rkllm3-server
Qwen2.5-3B-Instruct.rknn
Qwen2.5-3B-Instruct.weight
Qwen2.5-3B-Instruct.tokenizer.gguf
Qwen2.5-3B-Instruct.embed.bin
```

2. Start the Service

After making the file executable, start the service according to the model configuration:

```
./rkllm3-server \
  -m Qwen2.5-3B-Instruct.rknn \
  --weight Qwen2.5-3B-Instruct.weight \
  --vocab Qwen2.5-3B-Instruct.tokenizer.gguf \
  --embed Qwen2.5-3B-Instruct.embed.bin \
  --host 0.0.0.0 \
  --port 8080 \
  -c 768 \
  --n_predict 512 \
  --repeat-penalty 1.1 \
  --presence-penalty 1.0 \
  --frequency-penalty 1.0 \
  --top-k 1 \
  --top-p 0.8 \
  --temp 0.8
```

3. Check the Startup Status

After successful startup, the terminal displays information similar to the following:

```
load RKNN-LLM model: Qwen2.5-3B-Instruct.rknn
llm service is ready on http://0.0.0.0:8080
```

> If the file names, context length, or startup parameters differ for the current model or SDK version, use the corresponding model example and *RKLLM3_Server_User_Guide* as the authoritative reference.

6.3 Health Check

After starting the service, first use the health-check endpoint to confirm whether the service is ready:

```
curl http://localhost:8080/health
```

Health-check results are shown below:

HTTP Status	Meaning	Typical Response
503	Model is loading	`{"error": {"code": 503, "message": "Loading model", "type": "unavailable_error"}}`
200	Model loaded successfully	`{"status": "ok"}`

6.4 Send a Request Using curl

After the service is ready, use the OpenAI-compatible `/v1/chat/completions` endpoint to send a chat request:

```
curl --request POST \
  --url http://localhost:8080/v1/chat/completions \
  --header "Content-Type: application/json" \
  --header "Authorization: Bearer no-key" \
  --data '{
    "messages": [
      {"role": "system", "content": "You are a helpful assistant."},
      {"role": "user", "content": "Hello!"}
    ],
    "n_predict": 128
  }'
```

If `curl` is run on another host, replace `localhost` with the IP address of the device running `rkllm3-server`.

6.5 Send a Request Using the Python OpenAI Library

In addition to `curl`, you can use the Python `openai` library to send requests. The calling pattern matches the standard OpenAI API:

```
import openai

client = openai.OpenAI(
    base_url="http://localhost:8080/v1",  # Replace with the actual service
address
    api_key="sk-no-key-required"
)

completion = client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Write a limerick about python exceptions"}
    ]
)

print(completion.choices[0].message.content)
```

For streaming output, set `stream=True`:

```
completion = client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=[{"role": "user", "content": "Hello!"}],
    stream=True,
    extra_body={"n_predict": 128}
)

for chunk in completion:
    delta = chunk.choices[0].delta
    if delta.content:
        print(delta.content, end='', flush=True)
    if chunk.choices[0].finish_reason == "stop":
        break
print('')
```

`extra_body` supports the following non-standard OpenAI options:

Option	Description
`n_predict`	Number of tokens to predict for the current request
`id_slot`	Specifies the ID of the current slot (used with multiple sessions; default: 0)
`chat_template_kwargs`	Chat template parameters, such as `{"enable_thinking": False}` to control thinking output
`save_kvcache_path` / `load_kv cache_path`	Saves/loads KV Cache (the two options cannot be used at the same time)
`top_k`, `top_p`, `temperature`, etc.	Sampling parameters that override the startup defaults

6.6 Check the Response

A successful request returns a JSON response in a format compatible with the OpenAI API, for example:

```
{
  "id": "chatcmpl-xxx",
  "object": "chat.completion",
  "model": "Qwen2.5-3B",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "Hello! I am Qwen, a helpful AI assistant. How can I help you
today?"
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 20,
    "completion_tokens": 18,
    "total_tokens": 38
  }
}
```


For the first verification, confirm only the following:

- The HTTP request reaches the service (HTTP 200 is returned);
- The returned content is valid JSON;
- `choices[0].message.content` contains model-generated output.

7. Common Startup Issues and Next Steps

This chapter lists issues that may occur during the first run and provides troubleshooting methods. For the complete FAQ, model conversion issues, accuracy issues, performance issues, memory issues, and advanced deployment issues, refer to the troubleshooting chapters of *RKNN3_SDK_Development_Guide* and related documents.

7.1 Device Not Recognized

Development Board Host (`adb devices` shows no device):

1. Check whether the cable is connected correctly, reconnect the data cable, try another USB port on the computer, or replace the data cable.
2. When connecting the development board over USB, make sure only one adb server is running across the local computer and Docker container. For example, if you need to connect the development board from a Docker container, run `adb kill-server` in a terminal on the computer to stop the local adb server.
3. If adb is not installed: on Linux run `sudo apt install adb`; on macOS run `brew install android-platform-tools`.

```
# Linux example
sudo apt install adb
# macOS example
brew install android-platform-tools
```

PC Host (host `transfer_proxy devices` shows no device):

1. Check whether the USB cable supports data transfer, reconnect the cable, or try another port.
2. Confirm that RK182X is powered correctly and has finished booting.
3. Confirm that the platform-specific `rknn3_transfer_proxy` has been started on the host.
4. Check USB enumeration (environment-dependent): on Linux use `lsusb`; on macOS use `system_profiler SPUSBDataType`.
5. Run: `./rknn3_transfer_proxy/<platform>/rknn3_transfer_proxy devices` (Linux: `linux-x86_64`; macOS: `macos-arm64`).
6. You can still install and use `adb` on the development host for Development Board Host and other scenarios.

7.2 Device/Coprocessor Connection Issues

Because coprocessor mode uses a host-coprocessor architecture, the host connects to the coprocessor through PCIe / USB / Ethernet. Common topologies include Development Board Host `RK3588 ≒ PCIe / USB / Ethernet ≒ RK1820 / RK1828`, and PC Host `host (Windows / Linux) ≒ PCIe / Network / USB ≒ RK1820 / RK1828` and `host (macOS) ≒ USB ≒ RK1820 / RK1828`. If the host is visible but the coprocessor is not, try the following methods:

1. Check whether the PCIe/USB/Ethernet connection is correct, reconnect the device, or replace the data cable.
2. Run the following command and check whether the expected output is displayed:

```
./rknn3_transfer_proxy devices
# Reference output
List of ntb devices attached
0000:01:00.0        b98e6c51    PCIE
```

1. Confirm that `rknn3_transfer_proxy` has been installed and started correctly. If `rknn3_transfer_proxy` cannot be found, refer to 3.3 Install or Check the On-Board RKNPU3 Environment. For PC Host, run `./rknn3_transfer_proxy devices` on the **host**; for Development Board Host, run it on the board (for example via `adb shell`).

7.3 Toolkit Installation Verification Failed

First confirm that the expected Conda environment is active:

```
conda activate toolkit3
python --version
which python
```

Then run the minimal import test again:

```
python -c "from rknn.api import RKNN; print('RKNN3 Toolkit import OK')"
```

If it still fails, check the following first:

- Whether the Python version matches the `cpXX` value of the wheel file;
- Whether the requirements file matches the current Toolkit version;
- Whether the wheel comes from the current SDK release package;
- Whether the installation was performed in the wrong Python/Conda environment.

For complete installation and dependency troubleshooting, refer to *RKNN3_SDK_Development_Guide* and related documents.

7.4 `py_utils` Not Found During Model Conversion

When `ModuleNotFoundError: No module named 'py_utils'` appears, the Python environment variable is configured incorrectly. Set the environment variable according to the actual `rknn3-model-zoo` directory path.

For example, if `rknn3-model-zoo` is located at `/home/rockchip/rknn3-model-zoo`, configure it as follows:

```
export PYTHONPATH="/home/rockchip/rknn3-model-zoo:$PYTHONPATH"
echo $PYTHONPATH  # Confirm that the environment variable is set correctly
```

7.5 GRQ Quantization Failed

First check whether CUDA is available in the environment. External GRQ quantization is supported only in a CUDA environment. If the model is not supported, disable external GRQ quantization and use RKNN3 Toolkit for standard quantization.

7.6 Board-Side Runtime Version Mismatch

If the communication service and Runtime library versions do not match, initialization, PC-to-board inference, or on-board inference may fail. Check the versions separately:

```
# Runtime: Android
adb shell 'strings /vendor/lib64/librknn3_api.so | grep -i "librknn3_api version"'

# Runtime: Linux
adb shell 'strings /usr/lib/librknn3_api.so | grep -i "librknn3_api version"'
```

For RK182X, check:

```
# Android
adb shell '/vendor/bin/rknn3_transfer_proxy'

# Linux
adb shell '/usr/bin/rknn3_transfer_proxy'
```

For RK3572, check:

```
# Android
adb shell '/vendor/bin/rknn3-server -v'

# Linux
adb shell '/usr/bin/rknn3-server -v'
```

If the versions do not match, update the Runtime library and communication service using components from the same SDK release package. For the complete update procedure, refer to *RKNN3_SDK_Development_Guide* and related documents.

PC Host: on the host, verify that Runtime libraries in the install directory and `rknn3_transfer_proxy` from the same SDK package match:

Linux:

```
strings install/rknn3_model_test_Linux/lib/librknn3_api.so | grep -i 'librknn3_api
version'
./rknn3_transfer_proxy/linux-x86_64/rknn3_transfer_proxy
```

macOS:

```
strings install/rknn3_model_test_macos/lib/librknn3_api.dylib | grep -i
'librknn3_api version'
./rknn3_transfer_proxy/macos-arm64/rknn3_transfer_proxy
```

7.7 Documentation Navigation

After completing the first run, consult the single primary document for the relevant task. Do not continue searching for complete APIs or advanced configurations in this quick-start guide.

Topic to Continue With	Primary Document or Entry Point
RKNN3 software stack, component relationships, and complete development workflows for general-purpose models, LLMs, and VLMs	*RKNN3_SDK_Development_Guide*
Hardware, OS, and version support scope	*RKNN3_SDK_Development_Guide*
Complete RKNN3 Toolkit installation and model development instructions	*RKNN3_SDK_Development_Guide*
Toolkit methods, parameters, and return values	*RKNN3_Toolkit_Python_API_Reference*
RKNN3 C functions, structures, enumerations, and error codes	*RKNN3_Runtime_C_API_Reference*
RKNN3 Toolkit Lite Python methods and data types	*RKNN3_Toolkit_Lite_Python_API_Reference*
RKLLM3 Server deployment and advanced capabilities	*RKLLM3_Server_User_Guide*
RKLLM3 Server endpoints and request and response definitions	*RKLLM3_Server_User_Guide*
Accuracy, performance, memory analysis, and systematic troubleshooting	Troubleshooting chapters in *RKNN3_SDK_Development_Guide* and related documents

> **Note:** For related documents whose fixed file names are not specified in this guide, use the actual document names included in the current SDK release package.